



MODUL PERKULIAHAN

SISTEM KENDALI CERDAS

Kode Mata kuliah W132500026

Modul

Optimasi Kontrol dengan Algoritma

Abstrak

Genetic Algorithm mencari parameter controller tanpa memerlukan turunan objective. Metode ini cocok untuk objective nonlinier, diskrit, multi-puncak, atau berbasis simulasi, tetapi memerlukan evaluasi banyak kandidat dan desain fitness yang tidak dapat dieksploitasi secara keliru.

Disusun Oleh :

Dedik Romahadi, ST., M.Sc

Sub - CPMK

Sub-CPMK 5.4 — Mengoptimalkan parameter controller melalui kromosom, fitness, seleksi, crossover, dan mutasi



Modul

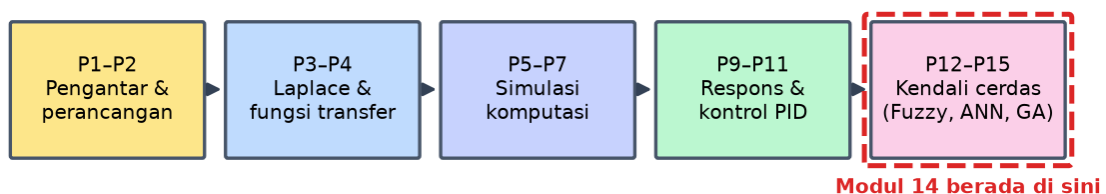
Interaktif:

<https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Sistem-Kendali-Cerdas/Modul/Modul-14.html>. Pada layar masuk, pilih tombol “👁 Mode Preview (Tanpa Login)” untuk membaca seluruh materi tanpa perlu NIM dan PIN. Untuk mengerjakan Tugas dan Forum, masuk sebagai Mahasiswa memakai NIM dan PIN.

PENDAHULUAN

Genetic Algorithm mencari parameter controller tanpa memerlukan turunan objective. Metode ini cocok untuk objective nonlinier, diskrit, multi-puncak, atau berbasis simulasi, tetapi memerlukan evaluasi banyak kandidat dan desain fitness yang tidak dapat dieksploitasi secara keliru. Gambar 1 mengilustrasikan gagasan ini.

Alur Mata Kuliah Sistem Kendali Cerdas



Gambar 1. Posisi Pertemuan 15 (Optimasi Kontrol dengan Algoritma Genetika) dalam alur mata kuliah Sistem Kendali Cerdas.

Modul ini disusun berlapis: pembahasan konsep, penurunan rumus yang dipakai, satu contoh terselesaikan, catatan salah kaprah yang sering terjadi, daftar periksa, lalu tugas terstruktur berupa sepuluh soal pilihan ganda dan lima belas soal komputasi. Versi interaktifnya menilai jawaban secara otomatis di server, sedangkan berkas ini dipakai untuk membaca dan mengerjakan secara luring.

Capaian Pembelajaran (Sub-CPMK)

- Sub-CPMK 5.4 — Mengoptimalkan parameter controller melalui kromosom, fitness, seleksi, crossover, dan mutasi

KETIKA PENCARIAN BERBASIS TURUNAN TIDAK DAPAT DIPAKAI

Metode pengoptimalan klasik bekerja dengan mengikuti gradien menuruni permukaan fungsi tujuan. Cara ini sangat efisien, namun menuntut tiga hal: fungsi tujuannya dapat

diturunkan, turunannya dapat dihitung dengan biaya wajar, dan permukaannya tidak memiliki terlalu banyak minimum lokal.

Pada penyetelan controller, ketiga syarat itu sering runtuh sekaligus. Fungsi tujuan biasanya berupa hasil simulasi, misalnya integral galat mutlak terhadap waktu, yang tidak memiliki bentuk analitik sehingga turunannya tidak tersedia. Kehadiran kejenuhan actuator dan pembatasan lain membuat permukaannya patah-patah.

Algoritma genetika tidak memerlukan turunan sama sekali. Ia hanya memerlukan kemampuan menilai seberapa baik sebuah kandidat, sehingga fungsi tujuan boleh berupa apa saja, termasuk hasil simulasi penuh dengan seluruh nonlinieritasnya.

Harganya adalah jumlah penilaian yang jauh lebih banyak. Bila satu penilaian menuntut simulasi yang lama, biaya totalnya bisa menjadi penghalang, sehingga perancangan fungsi tujuan yang murah dihitung menjadi bagian penting dari pekerjaan.

GA hanya perlu menilai, tidak perlu menurunkan | harganya: banyak penilaian

Empat Operator dan Peran Masing-Masing

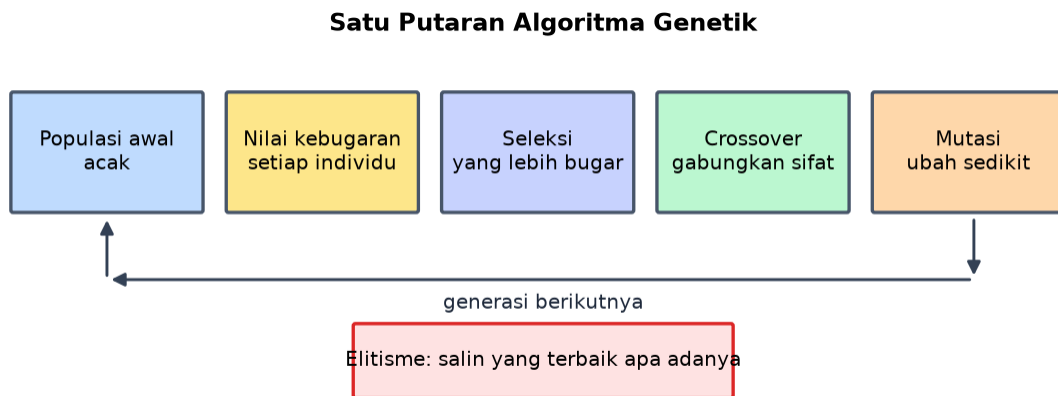
Seleksi menentukan kandidat mana yang berpeluang menurunkan sifatnya. Metode turnamen paling banyak dipakai karena sederhana dan tekanan seleksinya mudah diatur lewat ukuran turnamen. Tekanan yang terlalu besar membuat populasi cepat seragam dan pencarian berhenti dini.

Persilangan menggabungkan dua kandidat menjadi keturunan baru. Operator inilah yang bertanggung jawab menggabungkan bagian-bagian baik dari dua penyelesaian berbeda, dan menjadi pembeda utama algoritma genetika dari pencarian acak biasa.

Mutasi mengubah sedikit nilai secara acak. Perannya menjaga keragaman dan membuka kembali daerah pencarian yang sudah ditinggalkan populasi. Laju yang terlalu kecil membuat pencarian terjebak, sedangkan yang terlalu besar mengubahnya menjadi pencarian acak yang tidak terarah.

Elitisme menyalin beberapa kandidat terbaik langsung ke generasi berikutnya tanpa perubahan. Tanpa ini, penyelesaian terbaik yang sudah ditemukan dapat hilang akibat persilangan atau mutasi, sehingga kualitas terbaik dapat menurun antargenerasi, hal yang seharusnya tidak pernah terjadi.

seleksi → persilangan → mutasi → elitisme, berulang tiap generasi



Gambar 2. Satu putaran algoritma genetik beserta elitisme yang menjaga individu terbaik.

Gambar 2 memperlihatkan letak elitisme dalam daur: tanpa penyalinan itu, individu terbaik dapat hilang oleh crossover dan mutasi sehingga kebugaran terbaik justru memburuk antargenerasi.

Merancang Fungsi Tujuan yang Jujur

Fungsi tujuan adalah tempat seluruh keinginan perancang dinyatakan, dan algoritma akan mengejanya secara harfiah termasuk celahnya. Fungsi tujuan yang hanya menghukum galat akan menghasilkan penyetelan yang sangat agresif dengan sinyal kendali di luar kemampuan actuator, karena aspek itu tidak pernah dihukum.

Karena itu fungsi tujuan pada penyetelan controller umumnya menggabungkan beberapa suku: ukuran galat terhadap waktu, hukuman terhadap besar aksi kontrol, dan hukuman terhadap overshoot yang melewati batas. Bobot antarsuku menyatakan prioritas perancang secara eksplisit.

Ukuran galat yang dipilih memengaruhi hasil. Integral galat kuadrat menghukum penyimpangan besar jauh lebih keras sehingga menghasilkan respons agresif. Integral galat mutlak dikali waktu menghukum galat yang bertahan lama, sehingga cenderung menghasilkan penetapan yang cepat tanpa terlalu mengejar puncak awal.

Batasan keras sebaiknya ditangani terpisah dari bobot. Kandidat yang membuat sistem tidak stabil atau melanggar batas keselamatan lebih baik langsung diberi nilai sangat buruk, alih-alih sekadar dihukum, agar tidak pernah terpilih meskipun unggul pada aspek lain. Hubungan ini dirangkum dalam Persamaan (1).

$$J = w_1 \cdot ITAE + w_2 \cdot \text{integral}(u^2) + w_3 \cdot \text{penalti overshoot} \quad | \quad \text{pelanggaran keras: tolak} \quad (1)$$

Membaca Perilaku Pencarian

Grafik nilai terbaik terhadap generasi memberi banyak informasi. Penurunan yang cepat lalu mendatar menunjukkan pencarian sudah menemukan daerah yang baik; bila

mendatarnya terjadi sangat dini, biasanya keragaman populasi habis terlalu cepat akibat tekanan seleksi yang berlebihan.

Memantau keragaman populasi sama pentingnya dengan memantau nilai terbaik. Populasi yang seluruh anggotanya hampir sama tidak akan menghasilkan perbaikan lagi berapa pun generasi ditambahkan, karena persilangan antaranggota yang identik tidak menghasilkan apa pun yang baru.

Karena algoritma ini bersifat acak, satu kali jalan tidak membuktikan apa-apa. Praktik yang benar menjalankannya beberapa kali dengan benih acak berbeda, lalu melaporkan sebaran hasilnya, bukan hanya hasil terbaik yang kebetulan diperoleh.

Perlu diingat pula bahwa hasil terbaik menurut fungsi tujuan belum tentu terbaik menurut kebutuhan sesungguhnya. Kandidat pemenang wajib diperiksa ulang lewat simulasi penuh dan pengujian keadaan tepi sebelum dianggap layak.

pantau nilai terbaik DAN keragaman | jalankan beberapa kali, laporkan sebarannya

Menempatkan Algoritma Genetika pada Alur Perancangan

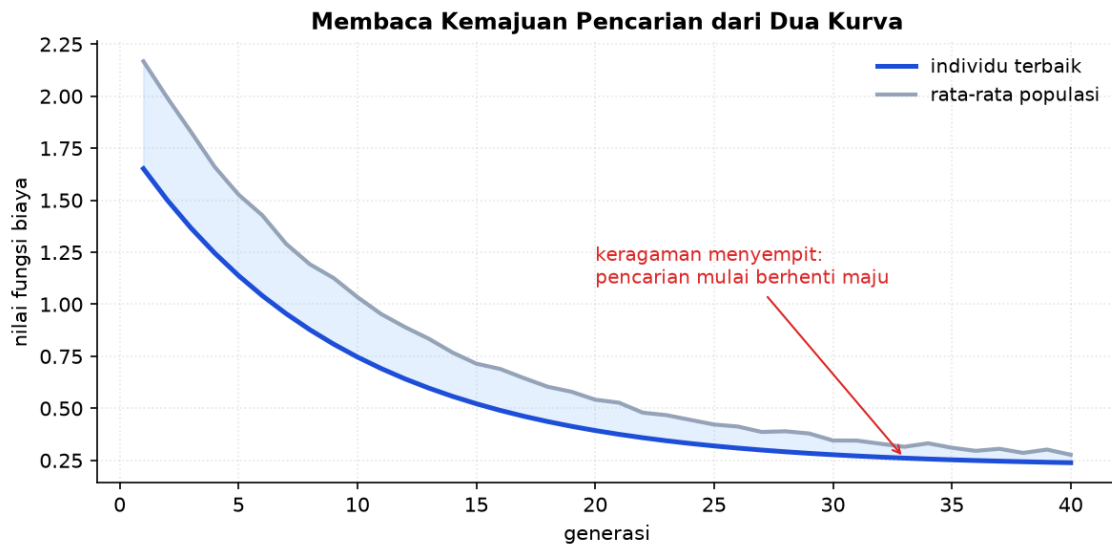
Algoritma genetika bukan pengganti pemahaman sistem. Ia bekerja paling baik ketika struktur controller sudah ditetapkan dengan benar dan yang tersisa hanyalah mencari nilai parameternya. Memakainya untuk menutupi struktur yang keliru hanya menghasilkan penyetelan terbaik dari pilihan yang buruk.

Penggunaan yang lazim mencakup penyetelan parameter PID pada plant yang nonlinier, pencarian letak dan bentuk fungsi keanggotaan pada controller fuzzy, serta penentuan bobot antartujuan yang saling bertentangan.

Karena penilaiannya berbasis simulasi, kualitas hasil dibatasi kualitas model. Model yang tidak memuat kejenuhan actuator akan menghasilkan penyetelan yang tampak unggul di komputer dan mengecewakan di lapangan, persoalan yang sama dengan simulasi pada umumnya, hanya diperbesar karena algoritma mengejar celah model secara sistematis.

Karena itu langkah akhir tetap sama seperti metode lain: nilai hasil pencarian diperlakukan sebagai titik awal, diperiksa terhadap batas actuator dan derau, lalu diuji bertahap pada perangkat dengan jalan keluar yang siap.

struktur ditetapkan lebih dahulu, GA mencari parameternya



Gambar 3. Kebugaran terbaik dan rata-rata populasi sepanjang generasi.

Gambar 3 memberi tanda kapan pencarian layak dihentikan: jarak kedua kurva yang menutup menunjukkan populasi telah seragam, sehingga generasi tambahan jarang memberi perbaikan.

Merancang Fungsi Kebugaran yang Jujur

Algoritma genetik akan memaksimalkan apa pun yang dituliskan pada fungsi kebugaran, termasuk hal yang tidak dimaksudkan. Karena itu merancang fungsi kebugaran adalah pekerjaan yang paling menentukan sekaligus paling mudah dilakukan secara keliru.

Kekeliruan yang lazim adalah menuliskan satu ukuran saja, misalnya jumlah kuadrat error. Optimasi terhadap ukuran tunggal itu menghasilkan controller yang cepat namun menuntut aksi kendali sangat besar, karena tidak ada satu pun suku yang menghukum pemborosan aktuator.

Perbaikannya menambahkan suku untuk setiap hal yang benar-benar dipedulikan: besar aksi kendali, lonjakan, dan pelanggaran batas. Bobot antarsuku menyatakan pertukaran yang dianggap wajar, dan menetapkannya memaksa perancang menyatakan pertimbangan yang biasanya hanya tersimpan sebagai kebiasaan.

Cara memeriksa kejujuran fungsi kebugaran cukup sederhana: jalankan optimasinya, lalu amati pemenangnya. Bila pemenangnya melakukan sesuatu yang secara teknis bernilai baik namun jelas tidak diinginkan, yang salah adalah fungsi kebugarannya, bukan algoritmanya. Bentuk ringkasnya dituliskan pada Persamaan (2).

$$J = w_1 \cdot \text{integral } e^2 + w_2 \cdot \text{integral } u^2 + w_3 \cdot \text{lonjakan} + \text{denda pelanggaran batas} \quad (2)$$

Menyetel Ukuran Populasi, Laju Mutasi, dan Elitisme

Tiga parameter memengaruhi perilaku pencarian lebih daripada yang lain, dan ketiganya berhubungan dengan pertukaran yang sama: seberapa lebar ruang dijelajahi dibanding seberapa cepat hasil terbaik dimanfaatkan.

Populasi yang besar menjaga keberagaman sehingga pencarian tidak cepat terjebak, namun setiap generasi menjadi mahal karena setiap individu harus dievaluasi. Pada persoalan kontrol dengan sedikit parameter, populasi beberapa puluh individu umumnya memadai, dan menambahnya lebih jauh jarang sepadan.

Laju mutasi yang terlalu kecil membuat populasi cepat seragam lalu pencarian berhenti maju; yang terlalu besar mengubah pencarian menjadi acak sehingga hasil baik tidak sempat diwariskan. Nilai di sekitar beberapa persen per gen menjadi titik awal yang wajar, dan menaikannya sementara berguna saat populasi terlihat mandek.

Elitisme menyalin beberapa individu terbaik apa adanya ke generasi berikutnya. Tanpa itu, solusi terbaik dapat hilang karena crossover dan mutasi, sehingga kebugaran terbaik justru dapat memburuk antargenerasi. Menyalin satu sampai dua individu sudah cukup; menyalin terlalu banyak membuat populasi cepat seragam.

populasi puluhan | mutasi beberapa persen | elitisme 1-2 individu

Menangani Kendala dan Solusi yang Tidak Sahih

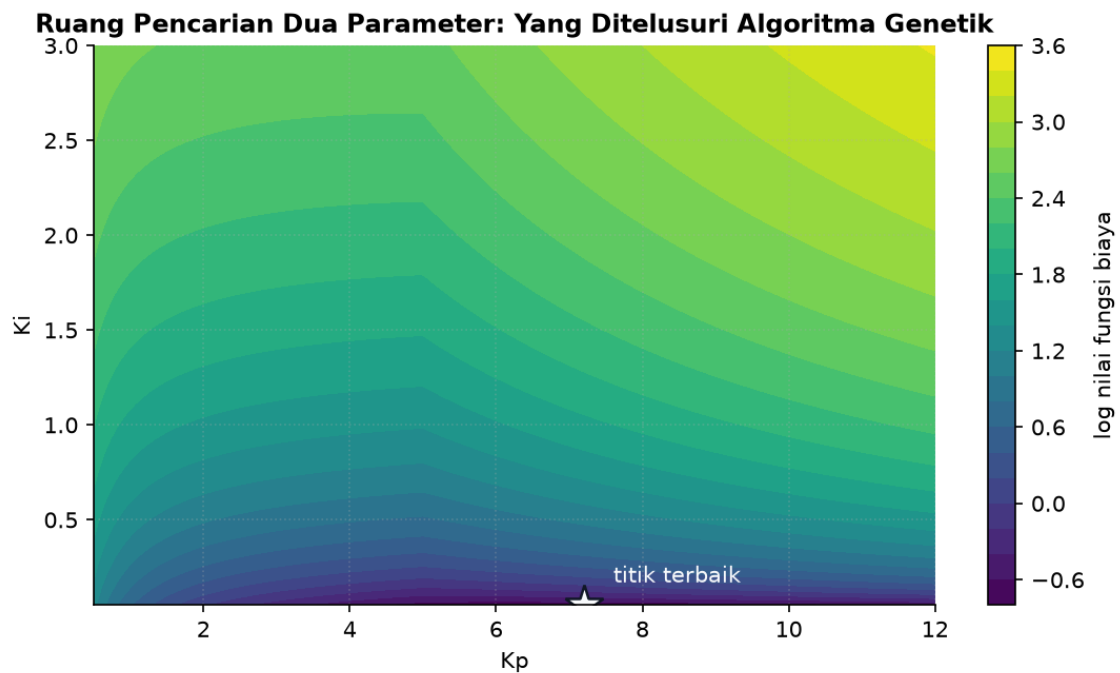
Persoalan kontrol hampir selalu memiliki kendala: penguatan harus positif, aksi kendali terbatas, sistem harus stabil. Algoritma genetik tidak mengenal kendala dengan sendirinya, sehingga penanganannya harus dirancang.

Cara pertama membatasi rentang setiap gen sehingga individu yang tidak sah tidak pernah tercipta. Cara ini paling aman dan paling murah, dan cocok untuk kendala sederhana seperti batas bawah dan batas atas parameter.

Cara kedua memberi denda pada fungsi kebugaran ketika kendala dilanggar. Cara ini diperlukan untuk kendala yang baru diketahui setelah simulasi, misalnya ketidakstabilan atau aktuator yang jenuh. Besar dendanya perlu cukup untuk membuat pelanggaran selalu kalah, namun tidak begitu besar sampai seluruh wilayah di dekat batas menjadi datar dan kehilangan arah perbaikan.

Cara ketiga memperbaiki individu yang tidak sah menjadi sah terdekat sebelum dievaluasi. Cara ini menjaga seluruh populasi tetap berguna, namun menambah pekerjaan dan dapat mengurangi keberagaman bila banyak individu diperbaiki ke titik yang sama.

batasi rentang gen → denda bertingkat → perbaikan, sesuai jenis kendala



Gambar 4. Permukaan fungsi biaya pada ruang dua parameter beserta letak titik terbaiknya.

Gambar 4 menjelaskan mengapa metode ini menarik untuk persoalan dengan banyak parameter: populasi menelusuri seluruh permukaan sekaligus, alih-alih menuruni satu lereng dari satu titik awal.

Dari Hasil Optimasi ke Sistem yang Berjalan

Parameter terbaik menurut simulasi belum tentu parameter yang layak dipasang. Jarak antara keduanya perlu dijembatani secara sadar, dan melewatkan langkah ini adalah penyebab paling umum kekecewaan terhadap hasil optimasi.

Pemeriksaan pertama menguji ketegaran. Parameter proses diubah sedikit di sekitar nilai nominalnya, lalu controller hasil optimasi diuji ulang. Bila kinerjanya jatuh tajam, hasilnya terlalu menempel pada model dan perlu dioptimasi ulang terhadap sekumpulan model, bukan satu model.

Pemeriksaan kedua membandingkan hasil optimasi dengan penyetelan manual yang baik. Bila selisihnya kecil, biaya dan risiko optimasi mungkin tidak sepadan, dan itu tetap merupakan temuan yang berguna. Bila selisihnya besar, ada baiknya dipahami dari mana keunggulannya berasal sebelum dipercaya.

Pemasangan sebaiknya bertahap: jalankan berdampingan tanpa kewenangan mengubah aktuator, bandingkan keluarannya dengan controller yang berjalan, baru serahkan kendali setelah perilakunya dipahami. Bertahap seperti ini mengubah kejutan menjadi pengamatan yang murah.

*uji ketegaran → bandingkan dengan penyetelan manual → jalankan berdampingan
→ serahkan kendali*

MENURUNKAN SATU GENERASI LENGKAP

Penurunan berikut menelusuri satu generasi pada populasi kecil, agar keempat operator terlihat bekerja berurutan. Persamaan (3) memadatkan aturan tersebut.

1. Populasi awal

empat kandidat dengan nilai tujuan $J = 12 ; 8 ; 20 ; 15$ (3)

Nilai lebih kecil berarti lebih baik. Rangkuman kuantitatifnya tertulis pada Persamaan (4).

2. Ubah menjadi kebugaran

$$F = 1/J = 0,0833 ; 0,125 ; 0,05 ; 0,0667 \quad (4)$$

Dibalik karena persoalannya meminimumkan.

3. Seleksi turnamen ukuran dua

adu acak berpasangan, ambil yang lebih bugar

Kandidat berkebugaran 0,125 paling sering terpilih. Hubungan ini dirangkum dalam Persamaan (5).

4. Persilangan aritmetik

$$anak = a \cdot induk1 + (1-a) \cdot induk2 \quad (5)$$

Dengan a acak antara nol dan satu untuk parameter bernilai nyata.

5. Mutasi

tambahkan derau acak kecil pada sebagian gen

Laju mutasi menentukan berapa banyak gen yang tersentuh.

6. Elitisme

salin kandidat terbaik apa adanya

Menjamin nilai terbaik tidak pernah memburuk antargenerasi.

7. Bentuk generasi baru

gabungkan elite dan keturunan sampai ukuran populasi terpenuhi

Siklus berulang sampai kriteria berhenti terpenuhi.

Perhatikan urutannya bukan kebetulan. Elitisme diterapkan terakhir justru agar melindungi kandidat terbaik dari kerusakan akibat persilangan dan mutasi yang baru saja terjadi. Tanpa

langkah itu, grafik nilai terbaik terhadap generasi dapat naik-turun, padahal seharusnya tidak pernah memburuk.

CONTOH TERHITUNG: SELEKSI DAN PERSILANGAN

Diketahui: Empat kandidat parameter Kp dengan nilai 2,0 ; 5,0 ; 8,0 ; 11,0 · Nilai fungsi tujuan berturut-turut J = 40 ; 12 ; 18 ; 55 (lebih kecil lebih baik) · Persilangan aritmetik dengan a = 0,3 ; elitisme menyalin satu kandidat terbaik

1. Urutkan menurut J

$$5,0 (12) ; 8,0 (18) ; 2,0 (40) ; 11,0 (55)$$

Kandidat Kp = 5,0 paling baik. Bentuk ringkasnya dituliskan pada Persamaan (6).

2. Tentukan elite

$$Kp = 5,0 \text{ disalin apa adanya } (6)$$

Elitisme menjamin nilai terbaik bertahan.

3. Turnamen pertama

$$\text{adu } 5,0 \text{ melawan } 11,0 \rightarrow \text{menang } 5,0$$

J = 12 lebih kecil daripada 55.

4. Turnamen kedua

$$\text{adu } 8,0 \text{ melawan } 2,0 \rightarrow \text{menang } 8,0$$

J = 18 lebih kecil daripada 40. Persamaan (7) memadatkan aturan tersebut.

5. Persilangan aritmetik

$$\text{anak} = 0,3 \cdot 5,0 + 0,7 \cdot 8,0 \quad (7)$$

Memakai kedua pemenang turnamen sebagai induk. Rangkuman kuantitatifnya tertulis pada Persamaan (8).

6. Hitung anak

$$= 1,5 + 5,6 = 7,1 \quad (8)$$

Nilai berada di antara kedua induknya. Hubungan ini dirangkum dalam Persamaan (9).

7. Mutasi anak

$$7,1 + 0,4 = 7,5 \quad (9)$$

Derau acak kecil ditambahkan untuk menjaga keragaman.

Jawaban. Generasi berikutnya memuat elite $K_p = 5,0$ dan keturunan $K_p = 7,5$, ditambah kandidat lain hasil turnamen berikutnya. Perhatikan bahwa persilangan aritmetik hanya menghasilkan nilai di antara kedua induknya, sehingga populasi cenderung menyusut ke satu titik. Mutasi itulah satu-satunya operator yang dapat membawa pencarian keluar dari rentang yang sudah ada, dan karena itu menolak laju mutasi hampir selalu membuat pencarian berhenti dini.

SALAH KAPRAH YANG SERING TERJADI

- Memakai algoritma genetika untuk menutupi struktur controller yang keliru — Hasilnya hanya penyetelan terbaik dari pilihan yang buruk. Struktur harus ditetapkan lebih dahulu.
- Menyusun fungsi tujuan yang hanya menghukum galat — Algoritma akan mengejanya secara harfiah dan menghasilkan penyetelan agresif dengan sinyal kendali di luar kemampuan actuator.
- Melupakan elitisme — Penyelesaian terbaik dapat hilang akibat persilangan atau mutasi, sehingga kualitas terbaik memburuk antargenerasi.
- Menolak laju mutasi — Persilangan aritmetik hanya menghasilkan nilai di antara induknya, sehingga populasi menyusut ke satu titik dan pencarian berhenti dini.
- Melaporkan satu kali jalan sebagai bukti — Algoritma ini bersifat acak. Beberapa kali jalan dengan benih berbeda beserta sebaran hasilnya jauh lebih jujur.

DAFTAR PERIKSA SEBELUM LANJUT

- ☐ Struktur controller ditetapkan sebelum pencarian parameter dimulai
- ☐ Fungsi tujuan memuat suku galat, usaha kontrol, dan hukuman pelanggaran batas
- ☐ Pelanggaran batas keras ditolak langsung, bukan sekadar dihukum
- ☐ Ukuran populasi, laju persilangan, dan laju mutasi dicatat
- ☐ Elitisme diterapkan agar nilai terbaik tidak pernah memburuk
- ☐ Keragaman populasi dipantau bersama nilai terbaik
- ☐ Pencarian dijalankan beberapa kali dengan benih berbeda
- ☐ Kandidat pemenang diperiksa ulang lewat simulasi penuh dan pengujian keadaan tepi
- ☐ Model yang dipakai menilai sudah memuat kejenuhan actuator

TUGAS

Tugas dikerjakan pada halaman modul interaktif agar penilaiannya otomatis. Bobotnya 50 poin: sepuluh soal pilihan ganda @1 poin, sepuluh soal komputasi tingkat dasar-menengah @2 poin, dan lima soal komputasi tingkat lanjut @4 poin. Soal komputasi dikerjakan dengan Python; yang dinilai adalah nilai yang dicetak, bukan cara penulisan programnya.

A. Pilihan Ganda (10 soal @1 poin)

1. Algoritma genetika tidak memerlukan turunan fungsi tujuan, sehingga cocok ketika...
 - A. Sistem berorde satu
 - B. Fungsi tujuan berupa hasil simulasi yang tidak punya bentuk analitik
 - C. Parameter hanya satu
 - D. Model plant sudah sangat akurat
2. Operator yang bertanggung jawab menggabungkan bagian baik dari dua penyelesaian adalah...
 - A. Seleksi
 - B. Elitisme
 - C. Mutasi
 - D. Persilangan
3. Peran mutasi adalah...
 - A. Mempercepat konvergensi
 - B. Menjamin nilai terbaik tidak memburuk
 - C. Menjaga keragaman dan membuka kembali daerah pencarian yang sudah ditinggalkan
 - D. Memilih induk terbaik
4. Tanpa elitisme, yang dapat terjadi adalah...
 - A. Penyelesaian terbaik hilang akibat persilangan atau mutasi sehingga kualitas terbaik memburuk antargenerasi
 - B. Pencarian menjadi terlalu lambat
 - C. Populasi menjadi terlalu beragam
 - D. Fungsi tujuan tidak dapat dihitung

5. Fungsi tujuan yang hanya menghukum galat akan menghasilkan...
 - A. Penyetelan paling seimbang
 - B. Penyetelan sangat agresif dengan sinyal kendali di luar kemampuan actuator
 - C. Penyetelan yang terlalu lamban
 - D. Kegagalan konvergensi
6. Pelanggaran batas keselamatan sebaiknya ditangani dengan...
 - A. Menambahkan bobot hukuman kecil
 - B. Menurunkan laju mutasi
 - C. Mengabaikannya karena jarang terjadi
 - D. Memberi nilai tujuan sangat buruk sehingga kandidat itu tidak pernah terpilih
7. Tekanan seleksi yang terlalu besar menyebabkan...
 - A. Populasi cepat seragam sehingga pencarian berhenti dini
 - B. Nilai terbaik memburuk
 - C. Fungsi tujuan menjadi tidak cembung
 - D. Persilangan tidak dapat dilakukan
8. Melaporkan hasil satu kali jalan sebagai bukti keunggulan tidak memadai karena...
 - A. Fungsi tujuan berubah tiap jalan
 - B. Perhitungannya selalu keliru sekali jalan
 - C. Algoritma bersifat acak sehingga hasilnya bervariasi antarjalan
 - D. Populasi awal selalu sama
9. Algoritma genetika bekerja paling baik ketika...
 - A. Struktur controller belum ditentukan
 - B. Struktur controller sudah ditetapkan dengan benar dan yang dicari hanya nilai parameternya
 - C. Model plant tidak tersedia sama sekali
 - D. Fungsi tujuan hanya memuat satu suku
10. Karena penilaian berbasis simulasi, kualitas hasil dibatasi oleh...
 - A. Kecepatan prosesor

- B. Jumlah generasi
- C. Kualitas model, terutama apakah kejenuhan actuator ikut dimodelkan
- D. Ukuran populasi

B. Komputasi Dasar-Menengah (10 soal @2 poin)

Parameter acuan: Populasi enam kandidat parameter K_p bernilai 1,0 ; 3,0 ; 5,0 ; 7,0 ; 9,0 ; 11,0 dengan nilai fungsi tujuan berturut-turut $J = 60 ; 28 ; 15 ; 22 ; 41 ; 70$ (lebih kecil lebih baik). Kebugaran diambil sebagai $1/J$. Seleksi memakai turnamen ukuran dua, persilangan aritmetik dengan $a = 0,4$, mutasi menambah 0,25, dan elitisme menyalin satu kandidat terbaik. Rinciannya dirangkum pada Tabel 1.

Tabel 1. Parameter acuan yang dipakai seluruh soal komputasi modul ini.

Parameter	Nilai
populasi	6 kandidat
J	60 ; 28 ; 15 ; 22 ; 41 ; 70
a persilangan	0,4
elitisme	1 kandidat

C1. Tentukan kandidat terbaik pada populasi awal, yaitu nilai K_p -nya. Cetak: K_p terbaik.

- PETUNJUK: Langkah: 1) kenali persoalannya meminimumkan sehingga J terkecil paling baik. 2) cari J terkecil di antara keenam nilai. 3) baca K_p yang bersesuaian.

C2. Hitung kebugaran kandidat terbaik. Cetak: F terbaik.

- PETUNJUK: Langkah: 1) cari J terkecil. 2) kenali kebugaran didefinisikan $1/J$ karena persoalannya meminimumkan. 3) hitung nilainya.

C3. Hitung kebugaran kandidat terburuk. Cetak: F terburuk.

- PETUNJUK: Langkah: 1) cari J terbesar. 2) hitung $1/J$. 3) bandingkan besarnya dengan kebugaran terbaik.

C4. Hitung rasio kebugaran kandidat terbaik terhadap terburuk. Cetak: rasio F .

- PETUNJUK: Langkah: 1) hitung kebugaran terbaik. 2) hitung kebugaran terburuk. 3) bagi keduanya.

C5. Hitung rata-rata nilai tujuan populasi awal. Cetak: rata-rata J .

- PETUNJUK: Langkah: 1) jumlahkan keenam nilai J . 2) hitung banyaknya kandidat. 3) bagi jumlah dengan banyaknya.

C6. Pada turnamen ukuran dua antara $K_p = 3,0$ dan $K_p = 9,0$, tentukan pemenangnya. Cetak: K_p pemenang.

– PETUNJUK: Langkah: 1) baca J kandidat pertama. 2) baca J kandidat kedua. 3) pilih yang J -nya lebih kecil.

C7. Pada turnamen antara $K_p = 5,0$ dan $K_p = 7,0$, tentukan pemenangnya. Cetak: K_p pemenang.

– PETUNJUK: Langkah: 1) baca J kandidat pertama. 2) baca J kandidat kedua. 3) pilih yang J -nya lebih kecil.

C8. Hitung keturunan hasil persilangan aritmetik kedua pemenang turnamen. Cetak: anak.

– PETUNJUK: Langkah: 1) ambil kedua pemenang sebagai induk. 2) tulis anak = $a \times \text{induk1} + (1-a) \times \text{induk2}$ dengan $a = 0,4$. 3) hitung nilainya.

C9. Hitung nilai keturunan setelah mutasi. Cetak: anak termutasi.

– PETUNJUK: Langkah: 1) peroleh nilai anak hasil persilangan. 2) ambil besar mutasi 0,25. 3) tambahkan keduanya.

C10. Hitung jumlah penilaian fungsi tujuan yang diperlukan pada generasi pertama. Cetak: penilaian.

– PETUNJUK: Langkah: 1) kenali setiap kandidat harus dinilai. 2) hitung ukuran populasi. 3) simpulkan jumlah penilaiannya.

C. Komputasi Lanjut (5 soal @4 poin)

C11 (Lanjut). Pencarian dijalankan 40 generasi dengan elitisme satu kandidat yang tidak perlu dinilai ulang. Hitung total penilaian fungsi tujuan. Cetak: total penilaian.

– PETUNJUK: Langkah: 1) hitung penilaian pada generasi pertama, yaitu seluruh populasi. 2) kenali pada generasi berikutnya satu elite sudah punya nilai. 3) hitung penilaian per generasi berikutnya. 4) hitung banyaknya generasi berikutnya. 5) kalikan keduanya. 6) tambahkan penilaian generasi pertama.

C12 (Lanjut). Satu penilaian menuntut simulasi selama 0,8 detik. Hitung total waktu pencarian, dalam detik. Cetak: total waktu (detik).

– PETUNJUK: Langkah: 1) hitung penilaian generasi pertama. 2) hitung penilaian per generasi berikutnya dengan elitisme. 3) hitung banyaknya generasi berikutnya. 4) jumlahkan menjadi total penilaian. 5) kalikan dengan lama satu simulasi. 6) nilai apakah waktunya masih praktis.

C13 (Lanjut). Fungsi tujuan gabungan disusun sebagai $J = 1,0 \times ITAE + 0,2 \times \text{usaha kontrol}$. Untuk kandidat dengan $ITAE = 12$ dan $\text{usaha} = 45$, hitung nilai J . Cetak: J .

– PETUNJUK: Langkah: 1) kenali kedua suku memiliki satuan berbeda sehingga bobot berperan menyetarakan. 2) ambil nilai $ITAE$. 3) kalikan dengan bobot pertama. 4) ambil nilai usaha kontrol. 5) kalikan dengan bobot kedua. 6) jumlahkan keduanya.

C14 (Lanjut). Kandidat yang melanggar batas keselamatan diberi nilai tujuan 1.000.000. Hitung rasio kebugaran kandidat terbaik terhadap kandidat pelanggar itu. Cetak: rasio kebugaran.

– PETUNJUK: Langkah: 1) cari J kandidat terbaik. 2) hitung kebugarannya sebagai $1/J$. 3) ambil J kandidat pelanggar. 4) hitung kebugarannya. 5) bagi kebugaran terbaik dengan kebugaran pelanggar. 6) simpulkan apakah pelanggar masih berpeluang terpilih.

C15 (Lanjut). Tanpa mutasi, rentang nilai populasi menyusut menjadi 0,85 kali setiap generasi. Dari rentang awal 10,0, hitung rentang setelah 10 generasi. Cetak: rentang akhir.

– PETUNJUK: Langkah: 1) kenali persilangan aritmetik hanya menghasilkan nilai di antara kedua induk. 2) karena itu rentang populasi tidak dapat membesar tanpa mutasi. 3) tulis rentang setelah n generasi sebagai rentang awal dikali faktor pangkat n . 4) masukkan faktor 0,85. 5) masukkan $n = 10$. 6) hitung nilainya.

DAFTAR PUSTAKA

Goldberg, D. E. (1989). Genetic algorithms in search, optimization, and machine learning. Addison-Wesley.

Eiben, A. E., & Smith, J. E. (2015). Introduction to evolutionary computing (2nd ed.). Springer.

Fleming, P. J., & Purshouse, R. C. (2002). Evolutionary algorithms in control systems engineering: A survey. Control Engineering Practice, 10(11), 1223-1241.

Skogestad, S., & Postlethwaite, I. (2005). Multivariable feedback control: Analysis and design (2nd ed.). Wiley.

Samad, T. (2017). A survey on industry impact and challenges thereof. IEEE Control Systems Magazine, 37(1), 17-18.